

Week 2: Grammar, Parsing, and a Minimal Interpreter

Aaron Foster

Southern New Hampshire University

CS 660: Advanced Program Language Theory

Prof. Shadha Tabatabai

July 19th, 2026

Part One: Define the Grammar

The language supports addition, subtraction, multiplication, and parentheses over positive integers. The following context-free grammar, written in BNF, defines the language. The terminal symbols are +, -, *, (,), and the digits 0 through 9; the start symbol is <Expr>.

$$\langle \text{Expr} \rangle ::= \langle \text{Expr} \rangle + \langle \text{Term} \rangle$$
$$| \langle \text{Expr} \rangle - \langle \text{Term} \rangle$$
$$| \langle \text{Term} \rangle$$
$$\langle \text{Term} \rangle ::= \langle \text{Term} \rangle * \langle \text{Factor} \rangle$$
$$| \langle \text{Factor} \rangle$$
$$\langle \text{Factor} \rangle ::= (\langle \text{Expr} \rangle)$$
$$| \langle \text{Number} \rangle$$
$$\langle \text{Number} \rangle ::= \langle \text{Digit} \rangle \langle \text{Number} \rangle | \langle \text{Digit} \rangle$$
$$\langle \text{Digit} \rangle ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9$$

Why the Grammar Rules Are Logically Consistent

The three-level layering of the grammar is deliberate: it encodes the conventional semantics of arithmetic directly into the syntax.

- **Precedence.** Multiplication appears at the $\langle \text{Term} \rangle$ level, below addition and subtraction at the $\langle \text{Expr} \rangle$ level, so $*$ always ends up deeper in the parse tree than $+$ or $-$. In $2 + 3 * 4$, the grammar forces the structure $2 + (3 * 4)$; no derivation produces $(2 + 3) * 4$.
- **Associativity.** The recursive rules are left-recursive ($\langle \text{Expr} \rangle ::= \langle \text{Expr} \rangle - \langle \text{Term} \rangle$), which makes subtraction group left-to-right: $8 - 3 - 2$ can only parse as $(8 - 3) - 2$.
- **Grouping.** The rule $\langle \text{Factor} \rangle ::= (\langle \text{Expr} \rangle)$ lets a parenthesized expression restart at the top of the hierarchy, which is exactly what parentheses mean: they override precedence.
- **Unambiguity.** A naive grammar such as $E ::= E + E \mid E * E \mid (E) \mid n$ is ambiguous; the string $2 + 3 * 4$ would have two distinct parse trees. The layered version admits exactly one parse tree per valid string, so structure (and therefore meaning) is fully determined by the syntax.

Formal Grammar Rules vs. Practical Expression Evaluation

Formally, the grammar answers only one question: which strings belong to the language, and what hierarchical structure do they have? A derivation from $\langle \text{Expr} \rangle$ produces a parse tree, and that is where the formal story ends. The grammar assigns no value to any expression; it defines shape, not meaning.

In practice, the interpreter carries out this specification in three distinguishable phases. First, a tokenizer flattens the raw input string into a stream of tokens. Second, a parser consumes the tokens left to right and builds an abstract syntax tree (AST). In a recursive-descent implementation, each nonterminal literally becomes a Python function (`parse_expression`, `parse_term`, `parse_factor`), so the grammar becomes the call graph of the running program. Third,

an evaluator performs a post-order traversal of the AST, applying one semantic rule per node type: an Add node's value is the sum of its children's values, a Mul node's value is their product, and so on. The formal grammar constrains structure, the evaluator supplies meaning, and correct output depends on the AST faithfully reflecting the derivation that the grammar defines.

Point of Divergence Between Theoretical Structure and Real-World Output

The clearest divergence involves the left-recursive rules. Formally, $\langle \text{Expr} \rangle ::= \langle \text{Expr} \rangle - \langle \text{Term} \rangle$ is the natural and correct way to specify left associativity. Practically, a top-down recursive-descent parser cannot execute this rule as written: `parse_expression` would immediately call `parse_expression` again without consuming a single token, recursing forever. The implementation must therefore rewrite the rule into an iterative (EBNF) form:

$$\text{Expr} ::= \text{Term} \{ (+ \mid -) \text{Term} \}$$

This rewritten form is where real-world output can silently diverge from the theoretical structure. The loop consumes operands left to right, and the implementer must manually fold them into a left-leaning AST. If the tree is instead built right-leaning - an easy mistake, and exactly what a naive right-recursive rewrite ($\text{Expr} ::= \text{Term} - \text{Expr}$) produces - the parser still accepts every valid string, but $8 - 3 - 2$ evaluates as $8 - (3 - 2) = 7$ instead of the correct $(8 - 3) - 2 = 3$. The language's syntax is preserved while its semantics are corrupted: the theoretical structure (the unique parse tree defined by the formal grammar) and the real-world output disagree, and no error is ever raised. This is a miniature version of the gap between formal and practical semantics - the specification and the implementation each look reasonable in isolation, and the discrepancy only surfaces on inputs that contain two or more left-associative operators.

Part Two: Grammar and a Parsing Diagram

Sample Expression and Diagram

The diagram below shows the complete parse tree for the sample expression $2 + 3 * (4 - 1)$, derived from the grammar defined in Part One. This expression was chosen deliberately because it exercises every structural feature the grammar encodes in a single input: operator precedence (+ versus *), grouping with parentheses, and the full $\langle \text{Number} \rangle$ and $\langle \text{Digit} \rangle$ derivation chain for each operand.

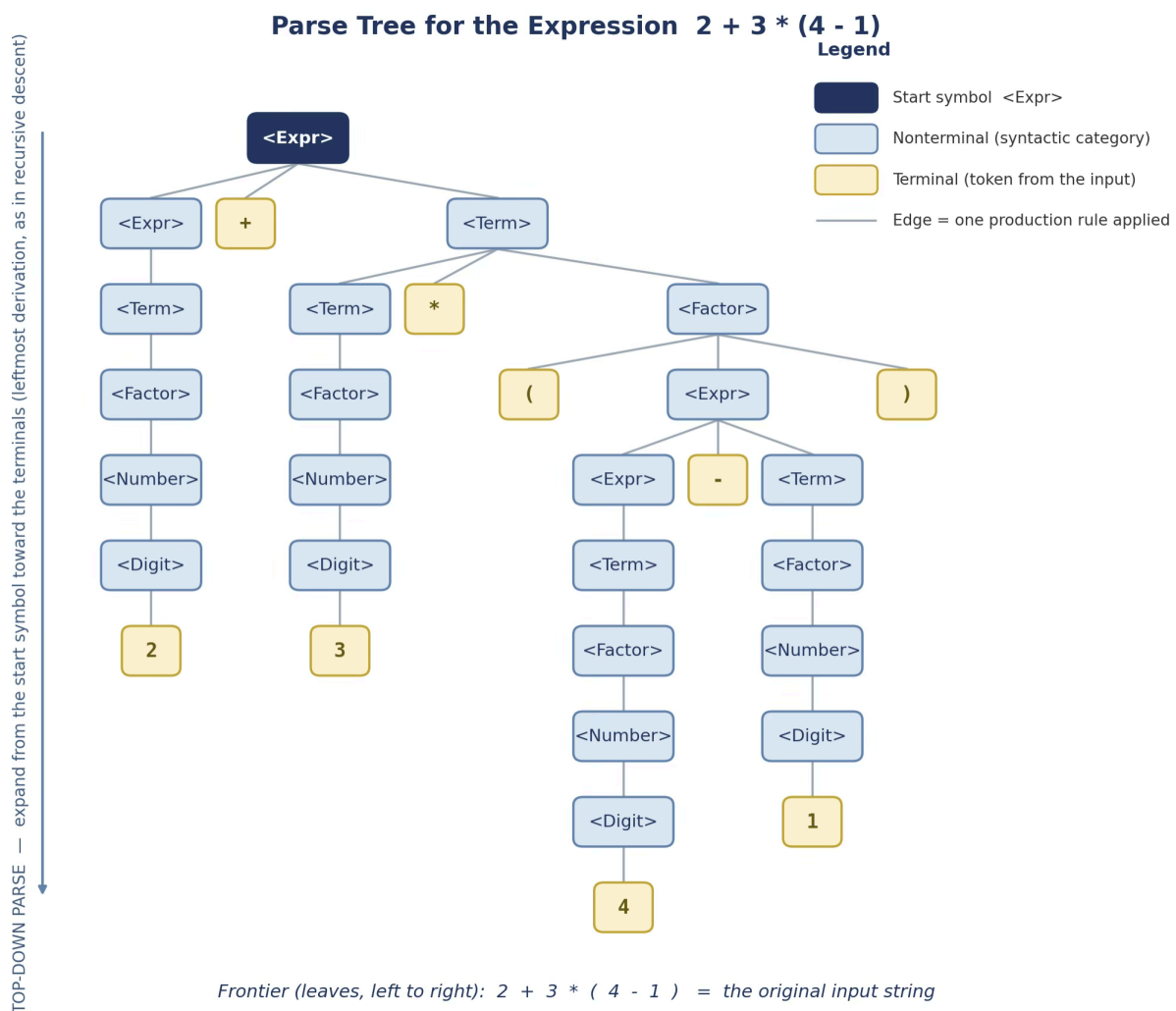


Figure 1. Parse tree for $2 + 3 * (4 - 1)$ under the Part One grammar, organized top-down.

Labeled Grammar Components

Every grammar component in the diagram is labeled and visually distinguished, as summarized in the legend:

- Start symbol. The root node $\langle \text{Expr} \rangle$ (dark blue) is the start symbol of the grammar; every derivation, and therefore every parse tree, begins there.
- Nonterminals. The light blue rounded nodes ($\langle \text{Expr} \rangle$, $\langle \text{Term} \rangle$, $\langle \text{Factor} \rangle$, $\langle \text{Number} \rangle$, $\langle \text{Digit} \rangle$) are the grammar's syntactic categories. Each one appears in the tree exactly where its production rules place it in the precedence hierarchy.
- Terminals. The amber leaf nodes (2, +, 3, *, parentheses, 4, -, 1) are tokens drawn from the input string. Terminals are exactly the nodes with no children: a parse is complete when every branch ends in one.
- Production applications. Each parent-to-children edge group represents one application of a production rule. For example, the root's three children ($\langle \text{Expr} \rangle$, +, $\langle \text{Term} \rangle$) are the right-hand side of $\langle \text{Expr} \rangle ::= \langle \text{Expr} \rangle + \langle \text{Term} \rangle$.
- Frontier. Reading the leaves left to right yields $2 + 3 * (4 - 1)$ - the original input - which confirms the tree is a valid derivation of the string.

The tree also makes the grammar's semantic encoding visible. The * operator sits strictly deeper in the tree than +, so multiplication is evaluated first, and the parenthesized subexpression $(4 - 1)$

re-enters the grammar at `<Expr>` inside a `<Factor>`, which is how parentheses override precedence.

Parsing Approach: Top-Down

The diagram is organized to reflect a top-down parsing approach. The start symbol `<Expr>` is placed at the top, and the tree grows downward: each level expands a nonterminal by replacing it with the right-hand side of one of its productions, until every branch terminates in a token. Read from top to bottom, the diagram traces a leftmost derivation of the input string - the same order in which a recursive-descent parser discovers the structure, beginning with a call to `parse_expression` and working toward the tokens. This is the approach implemented in Part Three. A bottom-up parser would instead start from the tokens at the leaves and reduce them upward toward the start symbol; the downward arrow on the left edge of the diagram marks the direction of derivation to make the top-down organization explicit.

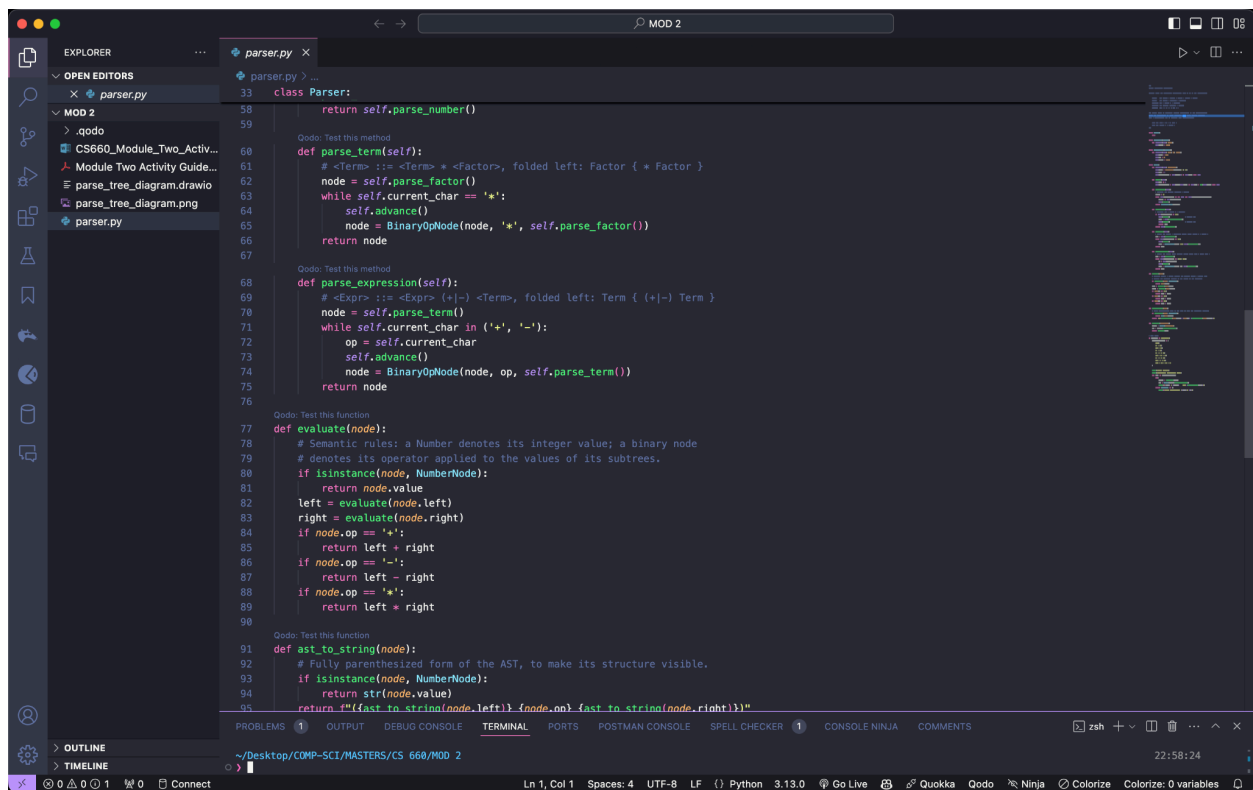
Part Three: Parser

Implementation Overview

The parser is implemented in Python (`parser.py`) as a recursive-descent parser, the top-down approach identified in Part Two. The correspondence between the grammar and the code is direct: each nonterminal becomes one parsing method. `parse_expression` implements `<Expr>`, `parse_term` implements `<Term>`, `parse_factor` implements `<Factor>`, and `parse_number` implements `<Number>` and `<Digit>` together by accumulating consecutive digit characters. The parser scans the input one character at a time (`current_char`, `advance`) and builds an abstract

syntax tree out of two node classes: `NumberNode`, which holds an integer value, and `BinaryOpNode`, which holds an operator and left and right subtrees.

As established in Part One, the left-recursive rules $\langle \text{Expr} \rangle ::= \langle \text{Expr} \rangle (+|-) \langle \text{Term} \rangle$ and $\langle \text{Term} \rangle ::= \langle \text{Term} \rangle * \langle \text{Factor} \rangle$ cannot be executed literally by a top-down parser, so `parse_expression` and `parse_term` implement the iterative form (Term { (+|-) Term } and Factor { * Factor }) with a while loop. Each iteration folds the node built so far into the left operand of a new `BinaryOpNode`, producing the left-leaning tree that the formal grammar requires.



```

class Parser:
    58     return self.parse_number()
    59
    60     Qodo: Test this method
    61     def parse_term(self):
    62         # <Term> ::= <Term> * <Factor>, folded left: Factor { * Factor }
    63         node = self.parse_factor()
    64         while self.current_char == '*':
    65             self.advance()
    66             node = BinaryOpNode(node, '*', self.parse_factor())
    67         return node
    68
    69     Qodo: Test this method
    70     def parse_expression(self):
    71         # <Expr> ::= <Expr> (+|-) <Term>, folded left: Term { (+|-) Term }
    72         node = self.parse_term()
    73         while self.current_char in ('+', '-'):
    74             op = self.current_char
    75             self.advance()
    76             node = BinaryOpNode(node, op, self.parse_term())
    77         return node
    78
    79     Qodo: Test this function
    80     def evaluate(self):
    81         # Semantic rules: a Number denotes its integer value; a binary node
    82         # denotes its operator applied to the values of its subtrees.
    83         if isinstance(node, NumberNode):
    84             return node.value
    85         left = evaluate(node.left)
    86         right = evaluate(node.right)
    87         if node.op == '*':
    88             return left * right
    89         if node.op == '-':
    90             return left - right
    91         if node.op == '+':
    92             return left + right
    93         if node.op == '/':
    94             return left / right
    95
    96     Qodo: Test this function
    97     def ast_to_string(self):
    98         # Fully parenthesized form of the AST, to make its structure visible.
    99         if isinstance(node, NumberNode):
   100             return str(node.value)
   101         return "(" + ast_to_string(node.left) + node.op + ast_to_string(node.right) + ")"

```

Evaluation via Semantic Rules

The evaluate function assigns meaning to the AST by structural recursion - a post-order traversal applying one semantic rule per node type:

`value(NumberNode n) = n.value`

`value(BinaryOpNode l '+' r) = value(l) + value(r)`

`value(BinaryOpNode l '-' r) = value(l) - value(r)`

`value(BinaryOpNode l '*' r) = value(l) * value(r)`

Because each rule is driven entirely by the shape of the tree, the evaluation order is determined by the structure the parser built, not by any separate precedence logic: $2 + 3 * 4$ evaluates to 14 because the parser placed the multiplication deeper in the tree, and $10 - 5 - 2$ evaluates to 3 because the subtractions were folded to the left. A helper (`ast_to_string`) prints each AST in fully parenthesized form - for example, $10 - 5 - 2$ prints as $((10 - 5) - 2)$ - making the tree structure visible in the program output and confirming that the parser avoided the associativity pitfall identified in Part One. All nine provided test cases produce correct results.

```
~/Desktop/COMP-SCI/MASTERS/CS 660/MOD 2
> python3 parser.py
Aaron Foster
Arithmetic Interpreter Test
42 = 42    AST: 42
2 + 3 = 5   AST: (2 + 3)
10 - 4 = 6   AST: (10 - 4)
3 * 7 = 21   AST: (3 * 7)
2 + 3 * 4 = 14   AST: (2 + (3 * 4))
(2 + 3) * 4 = 20   AST: ((2 + 3) * 4)
1 + 2 + 3 = 6   AST: ((1 + 2) + 3)
10 - 5 - 2 = 3   AST: ((10 - 5) - 2)
((1 + 2) * 3) - 4 = 5   AST: (((1 + 2) * 3) - 4)
```

How the AST Structure Corresponds to the Grammar Rules

The AST is a condensed image of the parse tree from Part Two: it keeps the structure the grammar derived while discarding everything that exists only to establish that structure.

- Binary productions become BinaryOpNode. Every application of $\langle \text{Expr} \rangle ::= \langle \text{Expr} \rangle (+|-) \langle \text{Term} \rangle$ or $\langle \text{Term} \rangle ::= \langle \text{Term} \rangle * \langle \text{Factor} \rangle$ in the derivation appears in the AST as exactly one BinaryOpNode, with the rule's left-hand nonterminal's subtree on the left and the $\langle \text{Term} \rangle$ or $\langle \text{Factor} \rangle$ subtree on the right.
- Number chains collapse into NumberNode. The derivation chain $\langle \text{Factor} \rangle \rightarrow \langle \text{Number} \rangle \rightarrow \langle \text{Digit} \rangle \rightarrow \text{digits}$ carries no structure of its own, so the AST collapses it into a single NumberNode holding the integer value.
- Parentheses disappear. The rule $\langle \text{Factor} \rangle ::= (\langle \text{Expr} \rangle)$ produces no AST node at all: `parse_factor` consumes the parentheses and returns the inner expression's subtree directly. The tokens can vanish because their entire purpose - overriding precedence - is already recorded in where the subtree sits. This is why $(2 + 3) * 4$ yields the AST $((2 + 3) * 4)$ with the addition below the multiplication, the mirror image of $2 + 3 * 4$.

In short, the grammar determines the shape of every AST the parser can build, and the evaluator's semantic rules consume exactly that shape - the practical implementation follows the formal specification, with the left-recursion rewrite as the one carefully managed departure.

Part Four: Reflection

The Structure of the Constructed Grammar

The grammar is organized as a three-level hierarchy - $\langle \text{Expr} \rangle$ above $\langle \text{Term} \rangle$ above $\langle \text{Factor} \rangle$ - with a small lexical layer ($\langle \text{Number} \rangle$, $\langle \text{Digit} \rangle$) underneath. What I found most instructive while constructing it is that this layering is not an organizational convenience: it is where the meaning lives. Precedence is never stated anywhere as a rule or a number; it is a consequence of multiplication being introduced one level deeper than addition, and associativity is a consequence of which side of each production recurses. Recursion appears in the grammar in two distinct roles: left recursion ($\langle \text{Expr} \rangle ::= \langle \text{Expr} \rangle + \langle \text{Term} \rangle$) expresses repetition of operators at the same precedence level, while the cycle through $\langle \text{Factor} \rangle ::= (\langle \text{Expr} \rangle)$ allows unlimited nesting depth from just five rules. A finite description generates an infinite language, and the shape of every sentence in that language is already decided by the shape of the rules.

How the Diagram Represents the Parsing Process

The diagram captures a process as a picture: it is a static record of the derivation that parsing performs over time. Each edge group in the tree is one application of a production rule, so reading the diagram from the root downward replays the parse step by step - $\langle \text{Expr} \rangle$ is expanded first, its children next, and so on, until every branch reaches a terminal. Two properties of the finished tree summarize the whole process. First, the frontier: reading the leaves left to right reproduces the original input $2 + 3 * (4 - 1)$, which certifies that the derivation actually generates the string being parsed. Second, depth encodes binding: the $*$ node sits below the $+$

node, and the parenthesized (4 - 1) sits deeper still, so the order in which the evaluator will later combine values is visible in the geometry of the diagram before any evaluation happens.

The Parsing Approach in the Grammar and the Diagram

Both the grammar and the diagram reflect a top-down approach. The diagram places the start symbol at the root and grows downward toward the tokens, tracing a leftmost derivation; the implementation is recursive descent, where that derivation becomes a chain of function calls beginning at `parse_expression`, and the current character decides which production to apply. The approach also shaped the grammar itself: the left-recursive rules had to be rewritten into iterative form to be executable top-down, which was the clearest lesson of the activity - the same language can require different grammar formulations depending on the parsing strategy, and the translation between them is precisely where a correct specification can quietly become an incorrect implementation. A bottom-up parser would have inverted the picture, starting at the leaves and reducing toward the start symbol, and could have used the left-recursive grammar as written.

AI Usage Acknowledgment

Portions of this assignment were completed with assistance from Anthropic's Claude (via Claude Code). The tool was used to help research and organize my draft paper against the rubric.

Anthropic. (2026). Claude [Large language model]. <https://claude.ai>